

Conditional Control Flow

Lecture 12

CS 1001

Kirk Duran

Today

- ▶ Use Boolean expressions to choose whether statements execute.
- ▶ Trace `if`, `if/else`, and `if/elif/else` statements.
- ▶ Distinguish chained conditionals from nested conditionals.
- ▶ Use `input()` to obtain user data and convert it to an appropriate type.
- ▶ Apply Python's truth-value rules when conditions are not explicitly Boolean.
- ▶ Use a predicate function call directly as a condition.

Key idea

Conditional control flow changes *which statement executes next* according to the truth value of an expression.

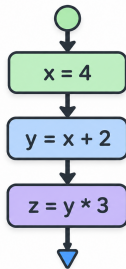
Part 1: From Boolean Values to Control Flow

Sequential Execution Has Only One Path

Sequential program

```
x = 4  
y = x + 2  
z = y * 3
```

- ▶ Each statement executes after the preceding statement.
- ▶ Tracing asks two questions: what is the current state, and what executes next?
- ▶ Until now, the second question usually had one answer.



A Conditional Introduces a Choice of Execution Paths

Definition

A **conditional statement** selects among possible execution paths according to the truth value of a condition.

Example

```
temperature = 82
if temperature > 80:
    print("Warm day")
```

- ▶ `temperature > 80` is evaluated before the body is selected.
- ▶ The condition produces a truth value.
- ▶ That result determines whether the indented statement executes.

Reminder: Conditions Are Expressions

Comparison

`age >= 18`

→ True or False

Compound Boolean expression

`age >= 18 and has_id`

- ▶ Arithmetic may be evaluated inside a condition.
- ▶ Comparisons produce Boolean values.
- ▶ `and`, `or`, and `not` combine or negate Boolean claims.
- ▶ Parentheses may make grouping explicit.

Key idea

The condition is evaluated using the same expression rules learned earlier; conditional control flow uses the resulting truth value.

Reminder: Python Also Uses Truthiness

Definition

Truthiness is Python's rule for interpreting a value in a Boolean context.

Common falsy values	Common truthy values
False	True
0, 0.0	nonzero numbers
""	nonempty strings
None	most other ordinary values

Useful diagnostic

`bool(value)` explicitly shows the truth value Python would use.

Tracing Now Requires State and Control

Program

```
x = 5
if x > 3:
    y = x + 1
z = x * 2
```

Step	Executed item	Bindings afterward
1	x = 5	x -> 5
2	evaluate x > 3	condition is True
3	y = x + 1	x -> 5, y -> 6
4	z = x * 2	x -> 5, y -> 6, z -> 10

Key idea

A trace must record not only values, but also which branch was selected.

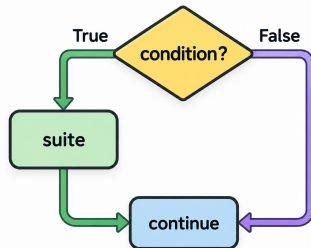
Part 2: The `if` Statement

The Simplest Conditional Is `if`

General form

```
if condition:  
    statement  
    statement
```

- ▶ Evaluate the condition.
- ▶ If its truth value is true, execute the indented suite.
- ▶ Otherwise, skip that suite.
- ▶ Continue with the first statement after the conditional.



The Colon and Indentation Define the Conditional Suite

Definition

A **suite** is the block of statements controlled by a compound statement such as `if`.

Example

```
if score >= 70:
    status = "passing"
    message = "Keep going"
print("Done")
```

- ▶ The colon introduces the suite.
- ▶ Both indented assignments belong to the `if`.
- ▶ `print("Done")` is outside the conditional and executes afterward regardless of the branch result.

A False Condition Skips the Entire Suite

Program

```
balance = 40
if balance >= 100:
    balance = balance - 100
    print("Purchase complete")
print(balance)
```

1. `balance >= 100` evaluates as `40 >= 100` -> False.
2. Neither indented statement executes.
3. Control continues at `print(balance)`.
4. The value of `balance` remains 40.

The Body May Contain Several Sequential Statements

Program

```
temperature = 95
if temperature > 90:
    message = "hot"
    water = 2
    print(message)
print("Finished")
```

- ▶ The condition selects whether the *entire* suite is entered.
- ▶ Once selected, statements inside the suite execute sequentially unless another control-flow construct changes that order.
- ▶ The first unindented statement marks the continuation point.

Trace Example: A Single if

Program

```
x = 6
y = 2
if x > y:
    x = x - y
    y = y + 1
result = x + y
```

Step	Action	State
1	x = 6	x -> 6
2	y = 2	x -> 6, y -> 2
3	x > y -> True	branch selected
4	x = x - y	x -> 4, y -> 2
5	y = y + 1	x -> 4, y -> 3
6	result = x + y	result -> 7

Prediction: Which Bindings Exist?

Program

```
points = 7
if points >= 10:
    bonus = 5
    points = points + bonus
final = points * 2
```

In class

Before running Python, determine:

1. whether the conditional suite executes;
2. whether `bonus` ever receives a binding;
3. the final values of `points` and `final`.

Key idea

A statement that is skipped cannot create or update a binding.

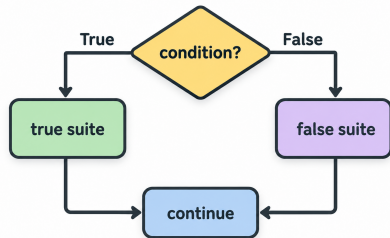
Part 3: Two-Way Selection with `if/else`

if/else Selects Exactly One of Two Suites

General form

```
if condition:  
    true_suite  
else:  
    false_suite
```

- ▶ A true condition selects the first suite.
- ▶ A false condition selects the else suite.
- ▶ The two suites are mutually exclusive for one execution of the statement.



Both Branches Rejoin at the Continuation Point

Program

```
score = 84
if score >= 70:
    status = "pass"
else:
    status = "not pass"
print(status)
```

- ▶ Exactly one assignment to `status` executes.
- ▶ After the selected branch finishes, both paths continue at `print(status)`.
- ▶ Designing both branches to establish the same required binding can simplify later code.

Trace Example: if/else

Program

```
x = 9
if x % 2 == 0:
    kind = "even"
else:
    kind = "odd"
length = len(kind)
```

1. $x \% 2 == 0 \rightarrow 1 == 0 \rightarrow \text{False}$.
2. The first suite is skipped.
3. The else suite binds `kind` $\rightarrow$ "odd".
4. `len(kind)` $\rightarrow$ 3, so `length` $\rightarrow$ 3.

Two Independent ifs Are Not the Same as if/else

Independent tests

```
if x > 0:
    print("positive")
if x < 10:
    print("small")
```

Both tests are evaluated.

Mutually exclusive alternatives

```
if x > 0:
    print("positive")
else:
    print("not positive")
```

Exactly one suite is selected.

Key idea

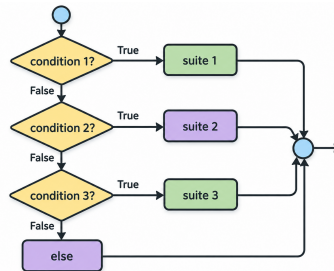
Program structure expresses a logical relationship among alternatives; indentation is not merely visual formatting.

Part 4: Chained Conditionals with `elif`

elif Represents Ordered Multi-Way Selection

General form

```
if condition1:  
    suite1  
elif condition2:  
    suite2  
elif condition3:  
    suite3  
else:  
    default_suite
```



Conditions in an `elif` Chain Are Tested Top to Bottom

1. Evaluate the `if` condition.
2. If it is truthy, execute that suite and skip the remainder of the chain.
3. Otherwise, evaluate the first `elif` condition.
4. Continue until one condition is truthy.
5. If no preceding condition is truthy, execute `else` when present.

Key idea

An `elif` chain selects the suite associated with the **first truthy condition**.

Order Matters When Conditions Overlap

Correctly ordered thresholds

```
if score >= 90:  
    grade = "A"  
elif score >= 80:  
    grade = "B"  
elif score >= 70:  
    grade = "C"
```

Why order is semantic

A score of 95 satisfies all three comparisons. The first true condition determines the selected branch.

For threshold classifications, testing from the most restrictive threshold downward is often the intended structure.

A Final `else` Can Make the Classification Exhaustive

Example

```
temperature = 58
if temperature >= 90:
    category = "hot"
elif temperature >= 70:
    category = "warm"
elif temperature >= 50:
    category = "cool"
else:
    category = "cold"
```

- ▶ Every numeric value reaches one branch.
- ▶ 58 fails the first two tests and satisfies `temperature >= 50`.
- ▶ The resulting binding is `category -> "cool"`.

Trace Example: A Chained Conditional

Program

```
x = 12
if x < 0:
    label = "negative"
elif x == 0:
    label = "zero"
elif x < 10:
    label = "small positive"
else:
    label = "large positive"
```

Evaluation sequence

12 < 0 -> False → 12 == 0 -> False → 12 < 10 -> False → else

Boundary Values Are the Best Tests for Chained Conditions

In class

For the grading chain below, predict the branch selected for scores 69, 70, 79, 80, 89, and 90.

Chain

```
if score >= 90:    grade = "A"  
elif score >= 80:  grade = "B"  
elif score >= 70:  grade = "C"  
else:             grade = "not passing"
```

Key idea

Boundary cases expose mistakes involving $<$ versus $\leq$, ordering, and missing ranges.

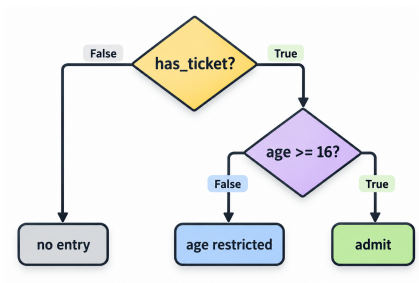
Part 5: Nested Conditionals

A Conditional Suite May Contain Another Conditional

Example

```
has_ticket = True
age = 15

if has_ticket:
    if age >= 16:
        status = "admit"
    else:
        status = "age restricted"
```



Nested and Chained Conditionals Represent Different Structures

Chained

```
if A:  
    ...  
elif B:  
    ...
```

B is considered only when A is false.

Nested

```
if A:  
    if B:  
        ...
```

B is considered only when A is true.

Key idea

Nesting expresses a dependency: reaching the inner decision depends on the outer path.

Trace Nested Conditionals from the Outside In

Program

```
x = 7
if x > 0:
    sign = "positive"
    if x % 2 == 0:
        kind = "even"
    else:
        kind = "odd"
result = sign + " " + kind
```

1. Evaluate outer condition: $7 > 0 \rightarrow \text{True}$.
2. Enter outer suite and bind sign $\rightarrow$ "positive".
3. Evaluate inner condition: $7 \% 2 == 0 \rightarrow \text{False}$.
4. Execute inner else: kind $\rightarrow$ "odd".
5. Continue after the outer conditional: result $\rightarrow$ "positive odd".

Choose the Structure That Matches the Logical Relationship

Relationship among tests	Typical structure
Perform something only when one claim is true	if
Choose between two alternatives	if/else
Choose the first matching case from an ordered set	if/elif/else
A second decision is meaningful only within one outer case	nested if

Design warning

Do not introduce nesting merely because it is available. Nest only when the logical dependency requires it.

Part 6: User Input, Truthiness, and Predicate Conditions

`input()` Reads One Line of User Input as a String

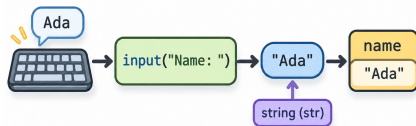
Definition

The built-in `input(prompt)` function displays the prompt, waits for a line of keyboard input, and returns the entered text as a `str`.

Example

```
name = input("Name: ")  
print(name)
```

Typing Ada produces the value "Ada".



Numeric Input Must Be Converted Before Numeric Comparison

Two-step form

```
age_text = input("Age: ")
age = int(age_text)
if age >= 18:
    print("Adult")
```

- ▶ If the user types 21, `age_text` is initially the string "21".
- ▶ `int(age_text)` evaluates to the integer 21 when the text is a valid integer representation.
- ▶ The comparison `age >= 18` is then a numeric comparison.

Key idea

User input is data; choose an appropriate representation before applying operations to it.

Trace Example: Input Followed by a Conditional

Program

```
number_text = input("Number: ")
number = int(number_text)
if number > 0:
    label = "positive"
else:
    label = "not positive"
print(label)
```

Suppose the user enters -3

```
number_text -> "-3" → number -> -3 → -3 > 0 -> False → label -> "not
positive"
```

Truthiness Matters with User Input

Program

```
name = input("Name: ")
if name:
    message = "Hello, " + name
else:
    message = "No name entered"
```

User enters Ada:

name -> "Ada"

nonempty string → truthy

User presses Enter immediately:

name -> ""

empty string → falsy

A Predicate Function Call Can Be Used Directly as a Condition

Definition

A **predicate function** is a function whose result represents a Boolean claim about its argument or arguments.

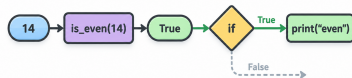
Example predicate

```
def is_even(n):  
    return n % 2 == 0
```

Using the predicate

```
value = 14  
if is_even(value):  
    print("even")
```

The call `is_even(value)` is an expression whose returned Boolean value becomes the condition result.



Integrated Trace: Input, Chaining, and a Predicate

Program

```
def is_even(n):  
    return n % 2 == 0  
  
text = input("Integer: ")  
number = int(text)  
if number < 0:  
    category = "negative"  
elif number == 0:  
    category = "zero"  
elif is_even(number):  
    category = "positive even"  
else:  
    category = "positive odd"
```

In class

Trace the program for user inputs -4, 0, 8, and 7. For each run, record the conditions evaluated, the first selected branch, and the final binding of category.

Key idea

Trace conditionals by separating three questions: **what does the condition evaluate to, which path is selected, and how does that path change program state?**